

AI GENERALIST ACCELERATOR · 14-DAY AI SPRINT

Day Four

AI Agents and RAG: Building a Customer Support Agent in n8n

A cleaned, detailed record of the live session and breakout briefing

Topics covered: The System Prompt · Jack, Jill, and RAG · Why Specific Context Wins · The Support-Agent Flowchart · Classify Then Branch · The RAG Pipeline in Full · Chunking, Embeddings, and Vectors · Building It in n8n · The Breakout Workbook

About This Session

Day four kept Jitesh Dugar in the chair for the second automation session and levelled up from yesterday's workflow to full AI agents, with Shasti hosting and setting the pace against a cohort now feeling the intensity of week one. His contract was the same, two and a half hours of undivided attention returned as knowledge worth bragging about, and his two declared habits held, too many questions and too many deliberate mistakes for the room to catch, both used all evening as teaching devices. The build was one level harder than day three by design, an email-answering customer support agent for a fictional bakery, and it carried the cohort into the single most-adopted AI pattern in industry, retrieval augmented generation, taught from first principles before a single node was placed. A housekeeping note opened the hour: yesterday's workbook API key had expired at 6 pm IST, so anyone still using it would hit errors, and today's workbook ships its own fresh key on the same one-day clock.

HOW TO USE THESE NOTES

Every resource lands on the platform under day four: the exported workflow JSON, the whiteboard screenshots, the RAG terminology handbook, and the bakery knowledge base. There are two workbooks, the in-session customer support triage bot on a chat trigger, and, under day four resources, an Additional Workbook rebuilding the exact RAG agent Jitesh demonstrated for anyone who wants the advanced version post-session.

Breakout rooms are never recorded, so a missed workbook goes to the Ask channel for a mentor reply within 24 hours, or to the next bonus office hour at 6 pm IST. Day 5 is a deep dive on advanced MCPs, building your own MCP server, with a preview of what days 6 and 7 hold.

The Missing Fourth Piece: System Prompts

The recap rebuilt yesterday's agent definition, a mix of an LLM to think, a memory to remember, and tools to execute, then added the piece day three had left out: the system prompt, also called system instructions or the system message, the guidelines that tell an agent how to behave, what it should do and what it must not. The analogy that made it stick was the job description, every employee has one telling them their remit, and a system prompt is an agent's job description. The live proof was ChatGPT itself: ask how to hack a neighbour's Wi-Fi and it refuses, because its own system prompt forbids helping with anything illegal or unethical, the same wall that meets how do I rob a bank. Three kinds of message round out the picture, taught later in the build but worth stating here, the system prompt is the built-in guideline only the agent's builder can see, the user prompt is the question or instruction you send, and the assistant message is the reply the agent returns, of which you control only the first two.

Jack, Jill, and the Shape of RAG

The evening's central concept arrived through two people of equal intelligence. Jill is handed a book and told to look it up before answering; Jack is told to answer from his own brain and never to say I don't know. On general knowledge, grams in a kilogram, both are right, but ask how to set a childproof lock on a specific microwave and only Jill, who can turn to the manual, answers correctly, while Jack, forbidden from admitting ignorance, confidently invents. What Jill actually does became the definition of the day: she turns to the relevant page, reads that page alongside the question, and gives the answer in plain English, three steps that are exactly retrieval augmented generation, retrieve the right page, augment it with the question, generate the answer. The point Jitesh hammered is why this matters so much, AI never says I don't know and will confidently hand you a wrong answer, and giving it a knowledge base is what forces accuracy, which is precisely why RAG customer support agents are among the most widely adopted AI solutions in any industry.

Why Specific Context Beats the Whole Book

A second exercise proved why RAG is not just dumping a document into the prompt. Two scenarios answered the same childproof-lock question, one feeding ChatGPT the entire microwave manual, the other only the single relevant page. The answers came out nearly identical, but the numbers diverged sharply: the whole book processed roughly forty thousand words where the page processed about five hundred, so the book ran slower and, because LLMs charge on input tokens, cost on the order of eighty times more for the same answer. The lesson, repeated as a rule, is always give your AI agent context only for the things it cares about, which is faster, cheaper, and more accurate all at once, and it is the whole reason the coming pipeline retrieves specific chunks rather than handing over the entire knowledge base.

The Support Agent, and the Flowchart First

The problem was made concrete: Bakehouse, an online bakery, normally handles customer email with a support team, until a great sales week dumps ten thousand emails in a day. At five minutes an email that is fifty thousand minutes, around 833 hours, roughly 34 days of one person's work, so the choices are hire more people or automate. Following his own first rule, Jitesh drew the flowchart before opening any tool. The trigger is an incoming email. The first action classifies it, customer support or not, because spam and marketing should be ignored rather than acknowledged, and an instant automated reply makes a we'll-get-back-to-you acknowledgement pointless. A branch splits on that result: not customer support does nothing, customer support flows into an agent that does four things, understand the customer's message, look up the knowledge base for the relevant policy, prepare a reply, and send it. The room correctly assigned the roles, the classify step is an LLM because classification is exactly what LLMs excel at, while the four-step responder must be an AI agent because sending a reply requires the ability to execute, which an LLM alone cannot do. A subtle point closed the design: you cannot branch directly off the classify LLM, you need the LLM to classify and then a separate branching node to route.

Building the Trigger and the Classifier

The build opened in a fresh workflow named Bakehouse Customer Support Agent with a Gmail trigger on message received, authenticated through sign-in with Google, its poll time left at every minute, Simplify left on for cleaner output, and the batch set to process ten emails per run at no extra cost since it only fires on new mail. When Fetch Test Event found no new email, Jitesh sent the bakery a sample complaint about a two-hour delivery delay, fetched it, and pinned the data so a page refresh would not lose it, with the reminder that pin data serves test executions only and never touches real incoming mail. The classifier came next, an OpenAI message-a-model node, and here the model-ladder logic from day three returned: classification is a simple task, so a small brain suffices, GPT-5.4 Mini chosen over the slower GPT-5 Mini. The prompt cast the model as an email intent classifier for Bakehouse that replies only with CS or not CS, with ordering, refunds, delivery tracking, and store timings counted as customer support and marketing or spam as not, and the first deliberate mistake surfaced immediately, the model replied understood, please provide the email, because the prompt was static and carried no email to judge. The fix was the core n8n mechanic again, switch from fixed to expression and drag the subject and snippet from the Gmail trigger into the prompt, after which it correctly returned CS. An If node then branched on whether that text equalled CS, the true branch confirming a genuine support email, with a note that Switch would work equally well and the CS-versus-not-CS shorthand being purely a personal choice, yes-no or full words work the same.

The RAG Pipeline, End to End

Before wiring the agent, Jitesh taught the whole RAG pipeline as two phases, using an illustration he had built with Claude rather than a live app. Phase one prepares the data. A source document is broken into small pieces by chunking; each text chunk passes through an embedding model that converts it into a vector, a numerical representation of text, because agents work far more comfortably with numbers than with raw text; and all those vector embeddings are stored in a specialised vector database, unlike a regular database built to hold exactly these embeddings. Phase two answers a question and mirrors the Jack-and-Jill steps. The user's question passes through the same embedding model into a query vector; a similarity search runs that query vector against the vector database to find the chunks most likely to hold the answer, the equivalent of turning to the right page; those chunks are augmented with the question and passed to a large language model, which, given specific pages rather than the whole book, generates the final answer. The bakery's own knowledge base, a nine-page policy handbook that he stressed could just as easily be ten thousand pages with the same accuracy, was the document about to go through this exact process, so that a question like my delivery was two hours late would retrieve the precise refund-and-goodwill policy rather than a guess.

The Terminology: Chunking, Splitters, Embeddings, Dimensions

After the break Jitesh walked the vocabulary the pipeline configuration would demand, all captured in the handbook. Chunking breaks a large document into manageable pieces because models have input limits and smaller pieces search faster; chunk size sets how big each piece is, where small chunks give high precision but little context and large chunks the reverse, so 500 to 1000 characters is the recommended starting range. Text splitters decide where the cuts fall, and the choice matters: a character text splitter cuts at a fixed count and will slice through the middle of a sentence, while a recursive character text splitter, used in more than ninety percent of cases, splits at natural boundaries in priority order, paragraphs, then sentences, then words, then raw characters, so each chunk stays whole and self-contained. Embeddings convert text to vectors, with similar words, cat and kitten, landing as similar numbers stored close together in the vector space. Dimensions are the length of that number list, and Jitesh gave the video-quality analogy: low dimensions like 384 are fast, cheap, and less accurate like 480p, high dimensions are the most accurate but slower and costlier, and a middle value around 768 to 1024 suits most work, with large dimensions reserved for legal or healthcare accuracy. Chunk overlap, his favourite, is the recap-of-the-previous-episode device: a small shared passage carried between consecutive chunks so context is not lost at the boundary, with 10 to 20 percent the recommended overlap.

Building It in n8n: Ingestion, Then the Agent

The implementation began with the data ingestion branch, and the vector database chosen was Pinecone, one of the most trusted, with a generous free plan, over n8n's own lighter built-in store. A Pinecone index named Bakehouse Knowledge Base was created at 1024 dimensions, the number that would have to match everywhere downstream. To get the document into n8n, Jitesh used a chat trigger on new chat event, enabled Allow File Uploads under its options, and uploaded the handbook; a Pinecone Vector Store node set to add documents then ingested it, authenticated with a Pinecone API key created exactly as the OpenAI key had been, using an OpenAI embeddings model whose dimensions were explicitly set to 1024 to match the index, a default data loader switched from JSON to binary because a file in automation is a binary, and a custom recursive character text splitter at 500-character chunks with 100-character overlap. Running it filled the vector database with readable chunks, all handled automatically once configured, and the whole ingestion branch was labelled a one-time step, rerun only when the knowledge base itself changes, updatable through the node's update-documents option.

The responder was assembled as an AI agent node on the CS branch, its user message set to the incoming email's subject and snippet dragged in as expressions, and its three required parts attached in turn. The brain was an OpenAI chat model on GPT-5.4 Mini, chosen over the slower GPT-5 Mini. The knowledge was wired not as raw access but as a Vector Store Question Answer Tool, itself a complete RAG pipeline in one node, pointed at the Pinecone index with its own matching OpenAI embeddings at 1024 dimensions and its own chat model, and Jitesh paused to explain why that second embedding model and chat model are needed alongside the agent's own: this tool independently takes a question from the agent, embeds it, searches Pinecone, augments the retrieved chunks, and returns a plain-English answer, a self-contained retrieve-augment-generate loop the agent simply calls. The action was a Gmail tool set to reply, with the message ID dragged dynamically from the trigger, another planted mistake corrected, and the reply content handed to the model itself through the let-the-model-define-this-parameter control, the magic AI button. Memory was added as simple memory keyed on the email thread ID rather than the message ID, since a new message ID is generated on every reply while the thread ID stays constant, so each customer conversation stays separate, with the email address offered as a workable alternative. The final missing piece was the system prompt, generated with Claude and pasted under the agent's options: Kelly from Bakehouse, instructed to always consult the knowledge base, to reply politely in well-formatted HTML, and to answer only from what the knowledge base returns.

Running It, and the Honest Misfires

Run live, the agent worked: for the delivery-delay email it looked up the policy, generated a warm reply from Kelly asking for the order number to investigate, and the RAG step's own log showed the agent had autonomously asked the knowledge base whether there was a compensation policy for delays over two hours. Published and tested end to end, the agent showed its rough edges honestly, which is what made the demonstration useful. A store-timings-and-ordering email first produced no reply at all, fixed by making the prompt explicit that it must always reply to every email. A follow-up answered the ordering methods correctly, website, WhatsApp, phone, and walk-in, and the two-hour delivery slots, straight from the handbook, but missed the store opening hours, prompting a further prompt tweak to always double-check the knowledge base before answering. When several questions were sent at once the mini model struggled, and the standing fix from day three applied, switch to a better model when quality slips. The recap closed the loop: the yellow ingestion branch runs once per knowledge-base change, the responder branch runs on every email, the classify step exists so the agent never wastes resources on non-support mail, and the two branches, though drawn on one canvas, are fully independent workflows with separate triggers.

THE MENTAL MODEL

RAG is Jack and Jill: retrieve the right page, augment it with the question, generate the answer, because specific context is faster, cheaper, and more accurate than the whole book. An agent needs four things, a brain, memory, tools, and a system prompt that is its job description; classification is an LLM job and replying is an agent job; ingest the knowledge base once into a vector database at a fixed dimension, then let a vector-store question-answer tool run the whole retrieve-augment-generate loop on every query; and when the small model misses, climb the ladder.

The Breakout, and What Comes Next

Shasti's wrap drew the honest line the breakout needed: the full RAG-and-vector-database build takes too long for a ninety-minute room, so the in-session workbook builds a different customer support triage bot, an agent that reads a query, judges its urgency, and for a high-urgency issue emails the customer an acknowledgement while escalating the ticket to the team. The room uses a chat trigger for feasibility, with an optional tab to rewire it to a Gmail trigger post-session, and its API key runs on the same clock, active until 6 pm IST the next day before revocation, with free self-made keys as the path afterward. For anyone wanting the exact agent Jitesh demonstrated, the Additional Workbook under day four resources rebuilds it in full, and a separate day three additional workbook, an expense-tracking system, answers the recurring request for a use case beyond the taught one, a nudge to explore the platform more fully. The rooms ran the usual app-only rule, and the sign-off set tomorrow's shape: bonus office hours at 6 pm IST, then a day five main session that is a deep dive on advanced MCPs, building your own MCP server, expanding on the connectors introduced on day one, with a look ahead to days six and seven. The Q&A; transcript for each day, it was noted, lives in the post-read section of the platform alongside these notes.

Key Takeaways

Stripped to its essentials, the session made the following points.

- **A system prompt is an agent's job description.** The fourth piece beyond brain, memory, and tools, it sets what the agent must and must not do, the same mechanism that makes ChatGPT refuse to help hack a Wi-Fi network.
- **RAG is Jack and Jill.** Give an AI a knowledge base and it retrieves the right page, augments it with the question, and generates a grounded answer, because an AI without a book will never say I don't know, it will confidently be wrong.
- **Specific context beats the whole book.** Feeding one relevant page instead of the entire manual gave the same answer at roughly an eightieth of the token cost and far faster, so always give an agent context only for what it cares about.
- **Classify with an LLM, act with an agent.** The support flow triages every email as CS or not with a cheap mini model, branches on the result, ignores non-support mail, and hands genuine tickets to an agent because only an agent can execute a reply.
- **Know the RAG vocabulary.** Chunk documents at 500 to 1000 characters with a recursive splitter that respects sentence boundaries and 10 to 20 percent overlap, embed the chunks into vectors, and store them in a vector database at a dimension, around 1024, that must match everywhere.
- **Ingest once, answer forever.** The Pinecone ingestion branch runs only when the knowledge base changes, while a single vector-store question-answer tool runs the entire retrieve-augment-generate loop on every customer query the agent receives.
- **Key the memory on the thread, not the message.** An email thread ID stays constant while message IDs change on every reply, so threading the agent's memory on it keeps each customer conversation cleanly separate.
- **Iterate the prompt, climb the model.** The live agent's misfires, no reply, a missed detail, a struggle with multiple questions, were each fixed by a more explicit prompt or a step up from the mini model, the honest reality of shipping an agent.

End of Day Four notes. Next: a deep dive on advanced MCPs — Day Five.